

Introduction to ONNX (Open Neural Network Exchange) Framework.

Abhishek D. Hirve

Dep.Computer Science

Università degli Studi di Verona, Italy

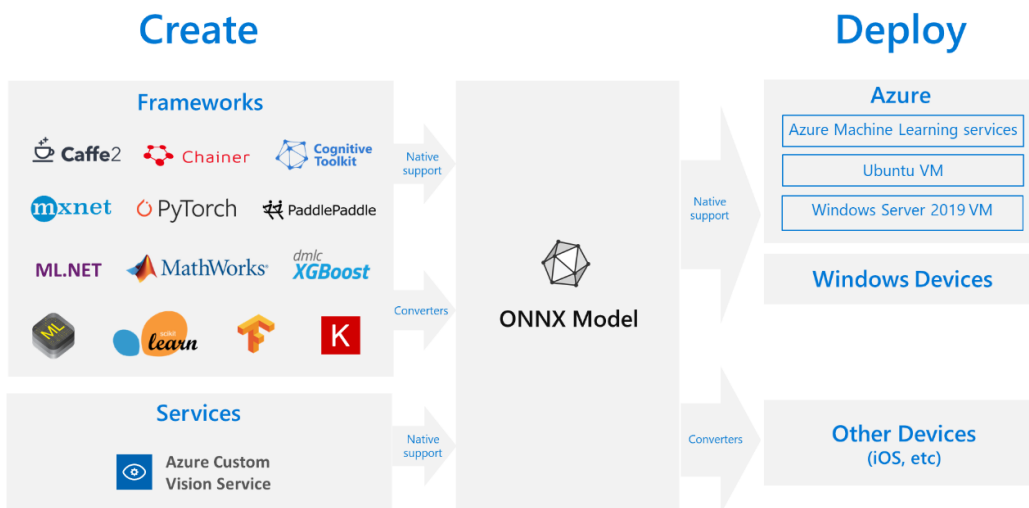
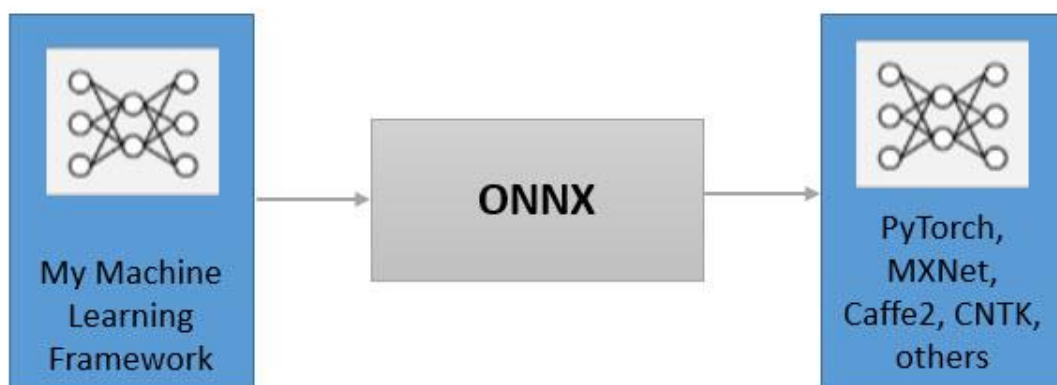
Abstract

ONNX (Open Neural Network Exchange) is a free and open-source system for expressing deep learning models and making them compatible across AI tools, platforms, and frameworks. It was founded in September 2017 by Facebook and Microsoft and has subsequently received backing from a number of organizations, including Amazon, IBM, Intel, Nvidia, and others. This article will present an overview of ONNX's characteristics and benefits, as well as its application cases and relevance in the field of artificial intelligence.

Keywords: Skit-Learn, ONNX , TF, GMM, pyplot.

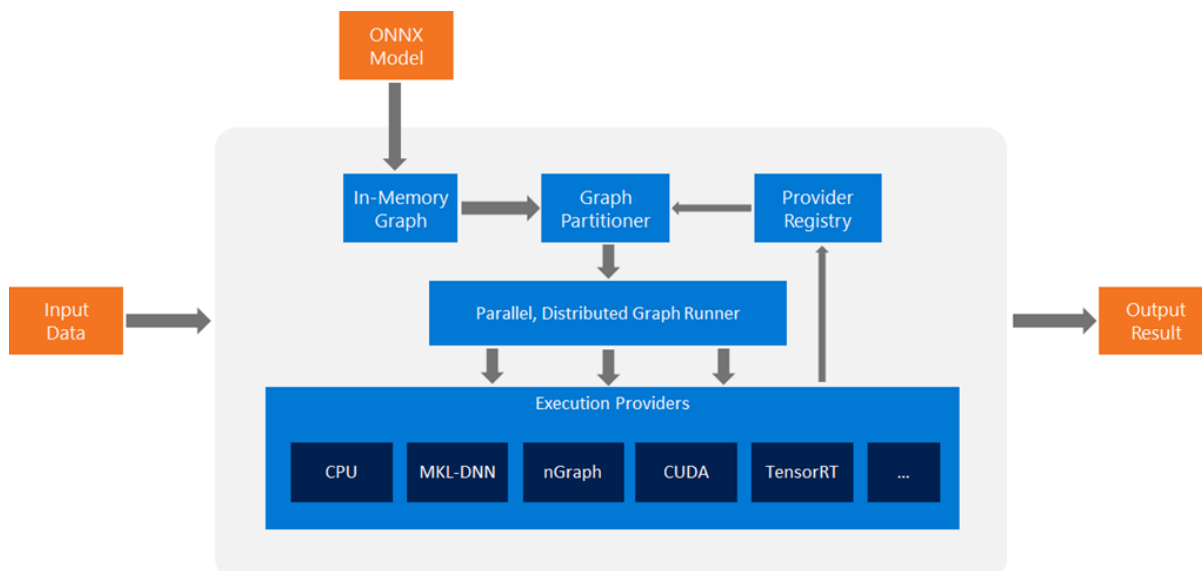
Introduction

Artificial intelligence (AI) has quickly emerged as a crucial technology in a wide range of sectors and applications, including computer vision, natural language processing, robotics, and healthcare. The capacity to express models in a format that is interoperable with many tools, platforms, and frameworks is a fundamental difficulty in AI development. ONNX is a free and open-source framework that provides a standard format for describing AI models, allowing for interoperability and portability across tools and platforms.



ONNX models in severa (16-02-2023) ONNX and Azure Machine Learning: Create and accelerate ML models. Available at: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-onnx>.

The following diagram shows the working mechanism of the ONNX engine:



➤ The working mechanism of ONNX involves the following steps:

- **Create or Train a Model:** The first step is to create or train a deep learning model using a framework such as TensorFlow, PyTorch, or Caffe2.
- **Convert the Model to ONNX Format:** Once the model is trained, it needs to be converted to the ONNX format. This can be done using one of the many conversion tools available such as ONNX-TF or ONNX-Caffe2.
- **Load the Model in the ONNX Runtime Engine:** After the model has been converted to ONNX format, it can be loaded into the ONNX runtime engine. The ONNX runtime engine is a lightweight and efficient runtime that provides support for running deep learning models on various hardware platforms.
- **Run Inference:** The final step is to run inference on the model. Inference refers to the process of using a trained model to make predictions on new data. The ONNX runtime engine provides a high-performance runtime that can run deep learning models on various hardware platforms such as CPUs, GPUs, and FPGAs.

ONNX Model -> Graph Construction -> Graph Optimization -> Graph Partitioning -> Operator Execution -> Model Output

The ONNX engine includes a runtime that allows deep learning models to be executed on a variety of hardware platforms. It comprises a collection of common operators supported by the vast majority of deep learning frameworks, as well as hardware-specific improvements that may be used to boost speed. The engine also contains quantization capability, which enables models to be compressed to minimize size and enhance inference performance.

- Features and Benefits ONNX provides several important features and benefits, including:
 - Interoperability: ONNX enables the exchange of models between different frameworks such as PyTorch, TensorFlow, and Caffe2, making it easier for data scientists and AI developers to work with the framework of their choice.
 - Portability: Models represented in ONNX format can be run on a variety of hardware, including GPUs, CPUs, and specialized AI accelerators, making it easier to deploy AI models in a wide range of environments.
 - Ease of use: ONNX provides a standard, reusable, and easily understandable format for representing AI models, making it easier to develop, debug, and maintain them.
- Use Cases ONNX is used in a variety of industries and applications, including:
 - Computer Vision: ONNX is used to represent models for object detection, segmentation, and classification tasks in computer vision.
 - Natural Language Processing: ONNX is used to represent models for tasks such as sentiment analysis, language translation, and text classification.
 - Healthcare: ONNX is used to represent models for tasks such as disease diagnosis, drug discovery, and medical imaging analysis.
 - Robotics: ONNX is used to represent models for tasks such as object recognition, motion planning, and control.

Conclusion

ONNX is a significant and growing AI framework with a huge and active contributor community. Because of its interoperability, portability, and ease of use, it is an excellent choice for generating and deploying AI models in a variety of applications. With increasing adoption and support from leading companies, ONNX is poised to play a larger role in the future of AI development. ONNX is a strong and versatile framework for expressing deep learning models in Python image processing. Its interoperability, portability, and simplicity of integration make it an excellent choice for data scientists and AI engineers working with Python deep learning models. With increasing acceptance and backing from top enterprises, ONNX is expected to become even more relevant in the future.

References

1. ONNX Website. (n.d.). ONNX | Open Neural Network Exchange. [online] Available at: <https://onnx.ai/> [Accessed 10 Feb. 2023].
2. ONNX GitHub Repository. (n.d.). onnx/onnx. [online] GitHub. Available at: <https://github.com/onnx/onnx> [Accessed 10 Feb. 2023].
3. ONNX Python Package. (n.d.). onnx/onnx-python. [online] GitHub. Available at: <https://github.com/onnx/onnx-python> [Accessed 10 Feb. 2023].
4. OpenCV Python. (n.d.). OpenCV Python. [Accessed 10 Feb. 2023]
5. PyTorch. (2020). PyTorch: An Open Source Deep Learning Platform. Retrieved from <https://pytorch.org/>
6. Zhu, T. et al. (2021). ONNX Runtime: Current Status and Future Directions. 28th ACM Symposium on Operating Systems Principles (SOSP '21), October 2021, Online Event, USA. DOI: 10.1145/3487552.3487574
7. Gupta, S. and Mohri, M. (2021). ONNX Transformer: Rethinking End-to-End Inference. arXiv preprint arXiv:2110.03485. Available at: <https://arxiv.org/abs/2110.03485> [Accessed 10 Feb. 2023].

Lab1

ONNX implementation in Python Image Processing

By following a few easy steps, ONNX may be used in Python image processing. Initially, in a framework such as PyTorch or TensorFlow, a deep learning model is trained. Using the relevant framework-specific tools, the model is then exported in ONNX format. Lastly, the ONNX model may be imported into Python and run on picture data using the ONNX Python module.

ONNX in Python Image Processing: ONNX is utilized in a number of Python image processing applications, including:

ONNX can be used to represent models for identifying things in photos like as humans, automobiles, or animals.

ONNX may be used to represent models in image segmentation. for dividing pictures into separate regions or objects, like in medical imaging or autonomous driving.

ONNX may be used to represent models in order to transfer the style of one image to another, providing artistic or creative effects.

➤ Limitations of ONNX

While ONNX offers many benefits, it also has some limitations. One of the main limitations is that it is limited to certain frameworks and hardware platforms. Additionally, ONNX is not able to support all types of machine learning models, as some models may require custom code to be written in order to be deployed in ONNX.

Another limitation of ONNX is that it is not able to support all types of data. Some data types, such as images, may require custom code to be written in order to be used in ONNX.

While ONNX provides significant benefits, it also has some limitations, including:

- **Model support:** ONNX supports a wide range of models, but not all deep learning models can be represented in ONNX format, which may limit its applicability in certain use cases.
- **Compatibility:** ONNX is compatible with a variety of deep learning frameworks, but not all versions of each framework are supported, which may cause compatibility issues when importing or exporting models.
- **Integration:** Integrating ONNX with other software tools, such as data processing pipelines, may require additional development and testing to ensure compatibility and performance.

Lab 2

converting skit-learn model in ONNX and display output using python.

In this example, we first create a simple scikit-learn model, which is a linear regression model that takes two features and predicts a single target variable. We then use the `onnxmltools.convert_sklearn()` function to convert the scikit-learn model to ONNX format and save it to a file called `model.onnx`.

Next, we load the ONNX model using the `onnxruntime.InferenceSession` function and create an input tensor with the same shape as the scikit-learn model's input. We then run inference on the input tensor using the ONNX model with `sess.run` to get the model's predictions.

Finally, we use `pyplot` to visualize the results. We first plot the training data as points on the graph, and then plot the model's predictions as red circles. We then call `plt.show()` to display the graph.

This is a simple example, but the same basic process can be used for more complex scikit-learn models as well. The key advantage of using ONNX is that it allows models to be easily ported between different deep learning frameworks and platforms, making it easier to deploy them in real-world production environments.

➤ Explanation:

This code trains a scikit-learn linear regression model on a two-dimensional dataset, then converts it to the ONNX format, saves the ONNX model to a file, loads it with ONNX Runtime, and uses it to make predictions on new input data. Finally, the results are visualized using the `pyplot` library.

➤ In more detail, the code performs the following steps:

A scikit-learn linear regression model is created and trained on a small dataset with two features, `X` and `y`.

The trained model is converted to the ONNX format using the `onnxmltools.convert_sklearn()` function. This function takes the scikit-learn model and an initial input type, which is specified as a list of tuples, where each tuple contains a name and a `FloatTensorType` object that specifies the shape of the input tensor.

The ONNX model is saved to a file named `model.onnx` using the `onnxmltools.utils.save_model()` function.

An input tensor is created with two rows of new input data, and the model is run on the input tensor using ONNX Runtime's `InferenceSession` class.

The output tensor is extracted from the `run()` method's return value by specifying the key `'0'`.

The input data, which consists of two rows of values for each of the two features, is plotted along with the output data. The input data is shown as a scatter plot, while the output data is shown as a line plot. The final plot is displayed using `pyplot`'s `show()` method.

- Plotting:

The resulting graph shows the input data points as blue dots and the output data points as red circles connected by a line. The line represents the predicted values for the given input data. Since the trained model was a linear regression model, the output data points lie on a straight line.

Lab 3

converting a scikit-learn GMM model to ONNX format and use it to predict the cluster assignments of new data points.

➤ Explanation :

This program demonstrates the use of ONNX (Open Neural Network Exchange) to convert a scikit-learn GMM (Gaussian Mixture Model) into an ONNX model and then use the ONNX Runtime to perform inference on new input data.

The program first creates a scikit-learn GMM model with 3 components and fits it on a 2-dimensional dataset X. It then converts this model into ONNX format, saves it to a file, and loads it into an ONNX Runtime session.

Next, the program creates a new input tensor of shape (2,2) and runs inference on it using the ONNX Runtime session. The resulting output tensor output_data is a 2-dimensional array of shape (2,3) which represents the predicted cluster membership probabilities for the input data.

Finally, the program uses matplotlib to visualize the original dataset X with its predicted cluster membership probabilities (denoted by color) and the new input data with its predicted cluster membership probabilities.

This program creates a 2D scatter plot using the matplotlib library. The plot consists of two sets of points:

The first set of points are the six points represented by the X array, which were used to train the Gaussian Mixture Model (GMM) created using the sklearn.mixture.GaussianMixture class. These points are plotted with colors based on the predictions made by the GMM using the fit_predict method.

The second set of points are the two points represented by the input_data array, which are used as input to the GMM after it has been converted to the ONNX format and loaded with the onnxruntime package. These points are plotted with colors based on the output generated by the GMM using the run method of the onnxruntime. InferenceSession class.

- Plotting:

The plot shows how the GMM makes predictions for points that were not part of the training data. The colors of the second set of points indicate which cluster the GMM assigns them to based on the input data.

****Note:** (All labs available in single notebook file name Project_final_ONNX.ipynb also, all lab files available individual folders for each Lab number label for folder.)